



DOCUMENTACIÓN DEL SISTEMA

1. Descripción General

- **Objetivo del sistema:**

Proveer una plataforma web (TMS - Transport Management System) enfocada en la optimización logística de rutas cafetaleras en la región de Amazonas. El sistema permite planificar rutas óptimas para el recojo de cargas en diferentes fincas, minimizando tiempos y costos de transporte.

- **Problema que resuelve:**

Resuelve el Problema del Agente Viajero (TSP) aplicado a la logística real, automatizando la asignación de rutas y vehículos. Elimina la planificación manual e ineficiente, reduciendo costos operativos, kilometraje innecesario y el consumo de combustible.

- **Arquitectura general:**

Arquitectura Cliente-Servidor (Monolítica). Backend desarrollado en Python (Flask) que procesa peticiones REST y sirve plantillas Jinja2, conectándose a una base de datos relacional PostgreSQL. Frontend interactivo utilizando HTML, CSS, Vanilla JS y la librería Leaflet para la visualización de mapas y rutas.

- **Flujo de funcionamiento del sistema:**

El usuario inicia sesión con su rol asignado. El despachador registra las cargas diarias. Luego, el conductor y/o administrador accede al mapa interactivo, selecciona los puntos de recojo y hace clic en "Calcular Ruta". El backend consulta los tiempos viales a la API de OSRM (o usa distancia de Haversine como fallback) y ejecuta un conjunto de algoritmos para resolver el TSP, mostrando la ruta óptima. El conductor puede "Iniciar" y "Finalizar" el viaje, registrándolo en la base de datos. El administrador monitorea en tiempo real todo el proceso, gestionando también la flota y los usuarios.

2. Tecnologías Utilizadas

- **Lenguaje de programación:** Python y JavaScript.
- **Versión exacta de Python utilizada:** Python 3 (Probado y compatible con 3.10+ según el estándar actual de las dependencias).

- **Frameworks utilizados:** Flask 3.1.3 (Backend Web).
- **Librerías principales y su propósito:**
 - `ortools` (9.15): Solucionador industrial de Google para problemas de ruteo vehicular y TSP.
 - `requests` : Para consumir APIs externas (OSRM).
 - `psycopg2-binary` : Driver adaptador para la conexión entre Python y PostgreSQL.
 - `pykml` : Parseo y lectura de archivos KML para extraer ubicaciones geométricas de fincas.
 - `Jinja2` : Motor de plantillas integrado con Flask para renderizar el Frontend.
 - `Leaflet.js` : Librería frontend para renderizar mapas interactivos.
 - `html2pdf.js` : Para exportar resúmenes a PDF en el frontend.
 - `xlsx` : Para exportación de datos a hojas de cálculo.
- **Dependencias importantes:** NumPy, Pandas, Werkzeug, Blinker.
- **Gestor de paquetes utilizado:** `pip` (con listado de dependencias en `requirements.txt`).

3. Base de Datos

- **Gestor de base de datos utilizado:** PostgreSQL.
- **Versión:** Compatible con PostgreSQL 13+.
- **Motor de almacenamiento:** El por defecto de PostgreSQL.
- **Estructura general:** Esquema relacional centralizado, asegurado mediante restricciones `UNIQUE` , y `FOREIGN KEYS` para integridad referencial.
- **Explicación del modelo de datos:**
 El sistema se sustenta en entidades principales: `usuarios` para el control de accesos, `precios_combustible` para tarifas económicas globales, `vehiculos` para la flota que depende de los precios de combustible, `historial_cargas` para la operación diaria, y `viajes_registro` que vincula a un usuario (Conductor) con un Vehículo para guardar la auditoría operativa del viaje.

4. Tablas de la Base de Datos

Nombre Tabla	Propósito	Campos Principales	Clave Primaria	Claves Foráneas	
precios_combustible	Almacena tarifas económicas dinámicas del combustible.	tipo_combustible , precio_galon_soles	id	Ninguna	1:
vehiculos	Catálogo de la flota de la empresa.	marca , modelo , placa , capacidad_kg , consumo_galon_km	id	tipo_combustible	N: p: 1: v:
historial_cargas	Almacena las cargas registradas por el despachador por día.	finca_nombre , fecha , cantidad_kg	id	Ninguna	Ni
usuarios	Control de accesos y perfiles de usuarios.	username , password_hash , nombre_completo , rol	id	Ninguna	1: v:
viajes_registro	Auditoría de viajes logísticos en tiempo real.	fecha , kg_totales , hora_inicio , hora_fin , estado	id	conductor_id , vehiculo_id	N: N:

5. Roles y Usuarios del Sistema

El sistema identifica estrictamente tres (3) tipos de roles mediante validaciones con el decorador `@role_required` en backend:

A. Administrador (`Admin`)

- **Función:** Supervisión global del sistema, gestión de flotas y de personal.
- **Permisos:** Totales. Puede crear/editar/eliminar usuarios y vehículos. Visualiza métricas analíticas.
- **Restricciones:** Ninguna.
- **Pantallas:** `/admin_dashboard` (Monitoreo, Gestión de Personal, Flota), `/dashboard` (Analíticas), `/rutas` (Mapa Logístico), `/cargas` (solo vista general).
- **Operaciones:** CRUD de Usuarios, CRUD de Vehículos, actualización de precios de combustible, monitoreo de viajes en tiempo real.

B. Conductor (`Conductor`)

- **Función:** Operario logístico que ejecuta el viaje físicamente.
- **Permisos:** Acceso al motor de rutas y visualización de viajes asignados.
- **Restricciones:** No puede gestionar cargas, usuarios, ni vehículos.
- **Pantallas:** `/rutas` (Mapa Logístico Principal), `/historial_rutas`.
- **Operaciones:** Seleccionar fincas, ejecutar cálculo del TSP, visualizar hoja de ruta, y accionar el control de viaje (Iniciar Ruta / Finalizar Ruta).

C. Despachador (`Despachador`)

- **Función:** Coordinador de campo que asigna los kilos a recoger diarios.
- **Permisos:** Puede insertar y eliminar cargas.
- **Restricciones:** Bloqueado totalmente de los módulos de mapeo (`/rutas`) para evitar distracciones operativas y no puede ver métricas financieras ni gestionar usuarios.
- **Pantallas:** `/cargas` (Gestión de Cargas).
- **Operaciones:** Añadir kilos a las fincas por día, ver inventario general de cargas.

6. Funcionalidades

Módulo de Autenticación

- **Objetivo:** Asegurar los endpoints y validar roles.
- **Flujo:** Login mediante `/login` POST. Comprobación del hash en DB. Almacenamiento del id y rol en `session` (cookie). Redirección condicionada según rol (Admin->Dashboard, Despachador->Cargas, Conductor->Rutas).
- **Archivos:** `app.py` , `templates/login.html` .
- **Tablas:** `usuarios` .

Módulo de Mapeo y Rutas

- **Objetivo:** Ejecutar la resolución del problema TSP y visualizarla.
- **Flujo:** Recepción de coordenadas vía Fetch API. Llamada al backend (`/api/calcular`). Generación de la matriz de distancias. Ejecución en paralelo de algoritmos. Comparación y ordenamiento de resultados. Renderización de la ruta ganadora con trazado vial en `Leaflet` .
- **Archivos:** `app.py` , `core_logistica.py` , `static/js/main.js` , `templates/index.html` .
- **APIs:** OSRM (Open Source Routing Machine).

Módulo de Control de Viajes

- **Objetivo:** Auditoría de viajes ejecutados por los conductores.
- **Flujo:** Conductor pulsa Iniciar, registra en la DB `hora_inicio` y estado `en_progreso` . El Frontend dispara un `setInterval` del reloj persistido en `localStorage` . Al finalizar, se manda POST a `/api/viaje/finalizar` actualizando la `hora_fin` .
- **Archivos:** `app.py` , `static/js/main.js` .
- **Tablas:** `viajes_registro` .

Módulo de Gestión de Flota y Precios

- **Objetivo:** Administrar los recursos logísticos y sus costos.
- **Flujo:** CRUD clásico vía endpoints `/api/vehiculos` y `/api/precios` desde la UI del panel de administrador.
- **Archivos:** `app.py` , `templates/admin_dashboard.html` .

- **Tablas:** `vehiculos` , `precios_combustible` .

7. Algoritmos Implementados

1. Google OR-Tools Routing (Voraz + Búsqueda Local)

- **Archivo:** `app.py`
- **Por qué se eligió:** Es el estándar de la industria. Maneja grandes grafos rápidamente.
- **Qué problema resuelve:** El TSP (Problema del Agente Viajero).
- **Cómo funciona:** Define un `RoutingIndexManager` , asigna una matriz de costos como callback. Utiliza la estrategia `PATH_CHEAPEST_ARC` para la primera solución, y la metaheurística `GUIDED_LOCAL_SEARCH` para refinar iterativamente.
- **Usuario:** Conductor, Administrador.
- **Complejidad:** Depende fuertemente de los parámetros de búsqueda, general $O(N^2)$ a polinomial según iteraciones.

2. Algoritmo Voraz (Greedy)

- **Archivo:** `core_logistica.py`
- **Cómo funciona:** Partiendo del nodo inicial, siempre elige el nodo no visitado más cercano en la matriz.
- **Ventajas:** Muy rápido $O(N^2)$.
- **Desventajas:** Rara vez encuentra el óptimo global; se usa de base comparativa.

3. Programación Dinámica (Held-Karp)

- **Archivo:** `core_logistica.py`
- **Cómo funciona:** Resuelve subproblemas de grafos con memorización (`memoization`) usando una máscara de bits para representar estados visitados.
- **Complejidad:** $O(N^2 2^N)$ en tiempo y $O(N 2^N)$ en memoria espacial.
- **Ventajas:** Encuentra la solución óptima absoluta.

4. Backtracking con Poda

- **Archivo:** `core_logistica.py`
- **Cómo funciona:** Búsqueda exhaustiva DFS en árbol, pero descarta ("poda") rutas

parcialmente armadas si su distancia actual ya superó a la mejor distancia encontrada.

5. Algoritmo de Fuerza Bruta

- **Archivo:** `core_logistica.py`
- **Cómo funciona:** Genera las permutaciones posibles con `itertools.permutations` calculando sus distancias.
- **Complejidad:** $O(N!)$ Factorial (Impracticable para $N > 11$).

6. Distancia Haversine (Matemática)

- **Archivo:** `core_logistica.py`
- **Propósito:** Actúa como algoritmo *Fallback* cuando la API externa de OSRM (vial) falla, calculando la distancia geodésica curva entre dos coordenadas terrestres.

7. Ordenamiento Merge Sort

- **Archivo:** Frontend (`static/js/main.js`) y Backend (`app.py`).
- **Cómo funciona:** Algoritmo Divide y Vencerás. Divide el arreglo en mitades, ordena y fusiona.
- **Propósito:** Ordenar las métricas resultantes del backend y ordenar alfabéticamente la lista de fincas en el frontend.
- **Complejidad:** $O(N \log N)$ (Tiempo).

8. Búsqueda Binaria

- **Archivo:** `static/js/main.js`
- **Propósito:** Búsqueda ultrarrápida del nombre de una finca en el panel lateral del mapa. Se aplica tras ejecutar el Merge Sort.
- **Complejidad:** $O(\log N)$ (Tiempo).

8. Flujo del Sistema

1. **Autenticación:** El usuario visita la raíz `/`, si no tiene cookie de sesión es redirigido a `/login`.
2. **Dashboard por Rol:**

- Si es Despachador -> Redirige a `/cargas` donde gestiona el peso de la mercancía.
 - Si es Administrador -> Redirige a `/admin_dashboard` para ver analíticas y auditar sistema.
 - Si es Conductor -> Redirige al mapa en `/rutas`.
3. **Módulo de Mapeo (Conductor):** Selecciona las fincas del día filtrándolas inteligentemente. Oprime "Calcular Ruta Óptima".
 4. **Respuesta Core:** El Backend dispara hilos múltiples o cálculos asíncronos y devuelve los resultados ordenados y la geometría del mapa.
 5. **Auditoría:** El conductor ejecuta físicamente el viaje controlándolo desde la App ("Iniciar/Finalizar"), lo que impacta inmediatamente los datos de monitoreo en el módulo del Administrador.

9. Estructura del Proyecto

```

/
├─ app.py                # Entry point de la aplicación y controladores REST de Flask.
├─ core_logistica.py     # Archivo núcleo con los algoritmos tradicionales de grafos y TSP.
├─ script_vehiculos.py   # Script utilitario o de simulación/batch.
├─ seed_usuarios.py      # Script semilla para inicializar cuentas en la Base de Datos.
├─ requirements.txt      # Lista oficial de librerías pip.
├─ database/
│   └─ schema.sql        # DDL y arquitectura física de las tablas y base de datos relacional.
├─ static/              # Archivos estáticos
│   └─ js/
│       └─ main.js       # Lógica frontend (Leaflet, DOM manipulation, timers, algoritmos JS).
└─ templates/           # Vistas (HTML + Jinja2)
    ├─ index.html        # Interfaz de mapa interactivo (Conductor).
    ├─ admin_dashboard.html # Panel modular (Administrador).
    ├─ cargas.html       # Panel de gestión (Despachador).
    ├─ login.html        # Vista de autenticación.
    ├─ historial_rutas.html # Vista del historial del conductor.
    └─ dashboard.html    # Analytics secundario.

```

10. Configuración del Proyecto

- **Variables de entorno:** La conexión está harcodeada internamente, pero el diseño

soporta variables estándar.

- **Configuración de la Base de Datos:** Host `localhost` , Puerto `5432` , Base de datos `tms_amazonas` , Usuario `postgres` .
- **Configuración del Servidor:** Servidor de desarrollo integrado de Flask (`Werkzeug`) levantado típicamente en puerto 5000.
- **Autenticación:** Se utiliza la librería de cifrado nativa `werkzeug.security` para el manejo de hashes y `flask.session` soportada por una Secret Key interna.

11. Dependencias

Librería	Versión	Uso dentro del Proyecto
<code>Flask</code>	3.1.3	Framework web principal. Enrutamiento y motor HTTP.
<code>psycopg2-binary</code>	N/A	Interfaz de base de datos para PostgreSQL.
<code>ortools</code>	9.15.6755	Proveedor del solucionador del motor de rutas avanzado (TSP).
<code>requests</code>	2.34.2	Peticiones HTTP sincrónicas al servicio externo de mapas (OSRM).
<code>pykml</code>	0.2.0	Parseo de geocódigos XML y lectura de mapas geográficos en formato KML.
<code>Werkzeug</code>	3.1.8	Sub-utilería de Flask; encriptación del <code>password_hash</code> (<code>generate_password_hash</code>).
<code>Jinja2</code>	3.1.6	Renderizado dinámico de las interfaces HTML.

12. Seguridad

- **Sistema de autenticación:** Sesiones basadas en Cookies cifradas del lado del cliente, firmadas mediante una `SECRET_KEY` .
- **Sistema de autorización:** Roles (`Admin` , `Conductor` , `Despachador`) protegidos a nivel endpoint a través de un decorador personalizado en Python llamado `@role_required` .
- **Manejo de contraseñas:** Cifrado unidireccional utilizando PBKDF2: SHA256 vía

Werkzeug Security.

- **Validaciones:** Validaciones en Frontend (requeridos, formatos de input, manejo de modales UI) e intercepción de Excepciones únicas (`psycopg2.errors.UniqueViolation`) en Backend para prevenir duplicados.
- **Protección contra ataques comunes:** Al usar SQLAlchemy/Psycopg2 con variables vinculadas parametrizadas (`%s`), el sistema está fuertemente protegido contra SQL Injection. Flask maneja protecciones integradas contra ataques genéricos en sus sesiones.

13. Resumen Técnico

El **TMS Amazonas** es una aplicación Full-Stack desarrollada en **Flask (Python)** y **PostgreSQL**. Destaca por su alta especialización técnica al resolver matemáticamente la logística terrestre utilizando **Google OR-Tools** combinado con algoritmos académicos tradicionales (Dinámica, Voraz). Implementa un esquema sólido de roles segmentando la operativa (Despachador), la ejecución en terreno (Conductor) y la analítica (Admin). Su UI se apoya en **Leaflet** y modales interactivos para dotar a la plataforma de reactividad. Un punto sumamente fuerte es su resiliencia mediante mecanismos *Fallback* geodésicos si la API de trazado vial externa fallase, manteniendo las operaciones a salvo.

14. Conclusiones

Actualmente, el sistema es totalmente operativo, robusto y se encuentra en una etapa madura que trasciende un simple visor de rutas. Se ha convertido exitosamente en una **Plataforma Multiusuario Transaccional** donde cada componente (Front, Back y BD) orquesta armónicamente. El Despachador alimenta el inventario, el Conductor interactúa con el motor algorítmico, y el Administrador goza de una gobernanza completa de la flota y monitoreo, cumpliendo un ciclo de vida logístico end-to-end de manera óptima y escalable.